A Project Synopsis on

VLSI Based Design and Implementation of Digital Baseband Processor for BLE

Submitted by

Prathma Jain (Roll no. 39)

Purva Patil (Roll no. 42)

Nixson Arumugam (Roll no. 43)

Abishek Jayaraj (Roll no. 44)

in fulfillment for the award of the degree

BACHELOR OF ENGINEERING

in

Electronics and Telecommunication Engineering

Under the Guidance of

Dr. Ravindra Chaudhari

St. Francis Institute of Technology, Mumbai

An Autonomous Institute, Affiliated to University of Mumbai

2025 - 2026

CERTIFICATE

This is to certify that Pratham Jain, Purva Patil, Nixson Arumugam, Abishek Jayaraj are the bonafide students of St. Francis Institute of Technology, Mumbai. They have successfully carried out the project titled "VLSI based Design and Implementation of Digital Baseband Processor for BLE" under the domain, VLSI , in fulfillment of the requirement of B. E. Degree in Electronics and Telecommunication Engineering of Mumbai University during the academic year 2025-2026. The work has not been presented elsewhere for the award of any other degree or diploma prior to this.

(Dr. Ravindra Chaudhari)

Internal Examiner

External Examiner

(Dr. Kevin Noronha)
EXTC HOD

(Dr. Deepak Jayaswal)
Principal

ACKNOWLEDGEMENT

We are thankful to a number of individuals who have contributed towards our final year project and without their help; it would not have been possible. Firstly, we offer our sincere thanks to our project guide, Dr. Ravindra Chaudhari for his constant and timely help and guidance throughout our preparation.

We are grateful to all project co-ordinators for their valuable inputs to our project. We are also grateful to the college authorities and the entire faculty for their support in providing us with the facilities required throughout this semester.

We are also highly grateful to Dr. Kevin Noronha, Head of Department (EXTC), Principal, Dr. Deepak Jayaswal, and Director Bro. Shantilal Kujur for providing the facilities, a conducive environment and encouragement.

Signatures of all the students in the group

(Pratham Jain)

(Purva Patil)

(Nixson Arumugam)

(Abishek Jayaraj)

Abstract

Bluetooth Low Energy (BLE) is a wireless communication technology that uses very little power and is commonly used in wearable and Internet of Things (IoT) devices to send data over short distances. It is made to allow communication that uses less energy while still making sure that data is sent reliably. This project shows how to design and build a BLE digital baseband transceiver using Verilog HDL. The transmitter part makes packets by adding a preamble, access address, header, payload, and CRC. Then it whitens the data using an LFSR and GFSK modulation. The receiver section does the opposite of the transmitter section, including demodulation, de-whitening, packet parsing, and CRC verification to get the original data back. The entire system is modeled and tested through simulation to make sure that data integrity, synchronization, and end-to-end communication are all correct. The results show that data was sent and received correctly, with accurate error detection. This work gives a detailed look at BLE baseband architecture and lays the groundwork for building wireless communication systems that use little power and work well.

Keywords: *Bluetooth Low Energy (BLE), Digital Baseband, Verilog HDL, GFSK Modulation, Data Whitening, CRC, LFSR, Transceiver Design, Wireless Communication, IoT.*

Contents

Acknowledgement **i**

Abstract **ii**

List of Figures **v**

List of Tables **vi**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Overview of Bluetooth Low Energy (BLE) 1

1.2 Baseband and Its Role in BLE 1

1.3 Motivation 1

1.4 Problem Statement and Scope 2

1.5 Organization of Project Report 2

2 Literature Review **3**

2.1 Introduction 3

2.2 A Low-Power BLE Baseband Transmission Data Processing Module (2024) 4

2.3 A Baseband Processor for Impulse Ultra-Wideband Communications (2005) 4

2.4 A 10 mW Bluetooth Low-Energy Transceiver with On-Chip Matching (2016) 5

2.5 Comparison of Reviewed Works 5

3 Theoretical Background and Design Methodology **6**

3.1 Theoretical Background 6

3.2 Design Methodology 7

3.2.1 BLE Transmitter 7

3.2.2 BLE Receiver 8
3.3 CRC Generation and Data Whitening Logic 13
3.3.1 CRC Generation using Fibonacci LFSR 13
3.3.2 Data Whitening using Galois LFSR 14
3.4 3.4 Gaussian Frequency Shift Keying (GFSK) Modulation 16
3.4.1 Principles of GFSK Operation 16
3.4.2 The Gaussian Shaping Filter 16
3.4.3 Significance in RTL Implementation 17
3.5 GFSK Demodulation 17
3.6 De-Whitening 18
3.7 Packet Structure and Parsing 18
3.8 CRC and Error Detection 18
3.9 Receiver Operation 18
3.10 Mathematical Derivation of the GFSK Shaping Pulse 18
3.10.1 Defining the Gaussian Filter in Frequency 21
3.10.2 The Inverse Fourier Transform 22
3.10.3 Convolution and the Error Function (erf) 22
3.10.4 Hardware Significance (RTL Implementation) 23
3.11 Phase Accumulator Calculations 23
3.11.1 Derivation of Frequency Deviation (Δf) 23
3.11.2 Phase Integral Derivation 24
3.11.3 RTL Significance: From Calculus to Hardware 24
3.12 Implementation Parameters and NCO Calibration 25
3.12.1 Clock Synthesis and PLL Scaling 25
3.12.2 Samples Per Symbol (SPS) 25
3.12.3 NCO Tuning Word Calculation 26
3.12.4 Simulation and Verification Results 26
3.13 Finite State Machine (FSM) Implementation 27
3.13.1 Transmitter FSM Logic 27
3.13.2 Receiver FSM Logic 28
3.14 Summary 29

4 Software Used and Simulation Results **31**

4.1 Software Tools Used 31

4.2 Simulation Results 32

4.3 Simulation and Performance Results 34

4.3.1 Performance Discussion 35

5 Conclusion and Future Scope **36**

5.1 Conclusion 36

5.2 Future Scope 36

References **37**

Appendix A: Project Timeline Chart 39

List of Figures

2.1 Generalized block diagram of a Bluetooth Low Energy (BLE) digital baseband architecture highlighting common functional modules reviewed in literature. 4

3.1 Process flow of BLE baseband processor. 7
3.2 Block Diagram of BLE Transmitter 8
3.3 Block Diagram of BLE Receiver 9
3.4 4-bit Fibonacci and Galois Linear Feedback Shift Register (LFSR) models used for CRC generation and data whitening respectively. 16
3.5 A Gaussian curve 17
3.6 Transmitter Operational State Diagram 28
3.7 Receiver Parser State Diagram 29

4.1 Transmitter Simulation Waveform 32
4.2 Receiver Simulation Waveform 32
4.3 Simulation Terminal Output. 33
4.4 Breakdown of the Packet. 33

List of Tables

2.1 Comparison of Reviewed Research Papers 5

3.1 Pin specifications for the BLE Baseband Processor Core. 10

3.2 Pin specifications for Transmitter Sub-blocks. 11

3.3 System Nomenclature and Mathematical Parameters 19

3.3 System Nomenclature and Mathematical Parameters 20

3.3 System Nomenclature and Mathematical Parameters 21

3.4 GTKWave Verification Summary 26

3.5 Transmitter State-Transition and Control Mapping 27

3.6 Receiver State-Transition and Control Mapping 28

3.7 Mathematical Core to RTL Logic Mapping 30

4.1 System Implementation and Verification Summary 34

List of Abbreviations

ASIC Application-Specific Integrated Circuit

ATT Attribute Protocol

BLE Bluetooth Low Energy

CMOS Complementary Metal-Oxide Semiconductor

CRC Cyclic Redundancy Check

DRC Design Rule Check

EDA Electronic Design Automation

FPGA Field Programmable Gate Array

FEC Forward Error Correction

GDSII Graphic Design System II (layout file format)

GFSK Gaussian Frequency Shift Keying

GATT Generic Attribute Profile

HDL Hardware Description Language

HCI Host Controller Interface

IoT Internet of Things

IP Intellectual Property

LFSR Linear Feedback Shift Register

LVS Layout Versus Schematic

L2CAP Logical Link Control and Adaptation Protocol

PDK Process Design Kit

PHY Physical Layer

PLL Phase-Locked Loop

RF Radio Frequency

RTL Register Transfer Level

TX Transmit Path

RX Receive Path

UWB Ultra-Wideband

VLSI Very Large-Scale Integration

VPD Verified Physical Design

VRC Verification and Routing Check

VCS Verilog Compiler Simulator

VDD Power Supply Voltage

Chapter 1

Introduction

1.1 Overview of Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) is a wireless communication standard that lets you send data over short distances and with low power. It works in the 2.4 GHz ISM band and uses Gaussian Frequency Shift Keying (GFSK) modulation to make it easy for devices like wearable sensors, smart home systems, and healthcare monitors to connect to each other. BLE is commonly used in Internet of Things (IoT) applications because it can keep communication going without using too much energy.

1.2 Baseband and Its Role in BLE

The baseband layer in BLE is the digital core that handles data before it is sent and after it is received. It controls the timing, packet formation, error detection, and data whitening in the link layer. The baseband connects the physical layer to higher protocol layers, making sure that data is handled and synchronized correctly. An efficient digital baseband processor is a key part of BLE hardware design because it affects the system's power use, processing speed, and reliability.

1.3 Motivation

The fast emergence of IoT has led to the need for power-efficient and effective wireless communication networks. Bluetooth Low Energy (BLE) technology has gained

popularity because it can facilitate reliable data transmission with very little energy consumption. But the existing BLE protocols have mainly been implemented as plug-and-play components, and little information about the design and operation of such devices has been available to the public. Consequently, there is less awareness of how data packets are transmitted and received through BLE basebands. In this context, the current project seeks to develop an implementation of the BLE baseband protocol using Verilog HDL code. Through this process, we will gain insights into packet generation, encoding, and error correction techniques.

1.4 Problem Statement and Scope

The growing popularity of IoT and wearables demands a need for power-efficient wireless communication systems. Even though BLE technology is extensively used in many applications, most of them are available as modules and do not provide much insight into the inner workings and optimization capabilities. This research aims to bridge that knowledge gap by creating a BLE baseband transceiver using Verilog HDL language. The emphasis will be on the major functions of the baseband transceiver, which include packet creation, CRC generation, data whitening with the help of LFSR, GFSK modulation and demodulation, and data de-whitening and parsing. The entire system will be tested and validated with simulation in order to ensure proper data exchange. However, the scope of this project will cover only the digital baseband section and not the radiofrequency (RF) front end.

1.5 Organization of Project Report

This project report is organized as follows:

Chapter 2: Presents the Literature Survey on the existing techniques.

Chapter 3: provides a brief explanation of Theoretical Background and Design Methodology.

Chapter 4: is dedicated to the Software Tools used and Simulation Results.

Chapter 5: presents the Conclusions and Future Scope for this project.

Chapter 2

Literature Review

2.1 Introduction

The Bluetooth Low Energy technology allows short range, low energy wireless communication between IoT devices. The baseband processor performs digital signal processing tasks like packet generation, cyclic redundancy check (CRC), data whitening, and synchronization. Various scholars have attempted to enhance the baseband modules in order to ensure improved performance, lower power consumption, and area. The selected scholarly articles collectively discuss the optimization methods that have been employed in the designs of digital baseband circuits, low energy systems, and integrated BLE transceivers. A general model of a BLE baseband circuit design as covered in the papers is illustrated below as Fig. 2.1.

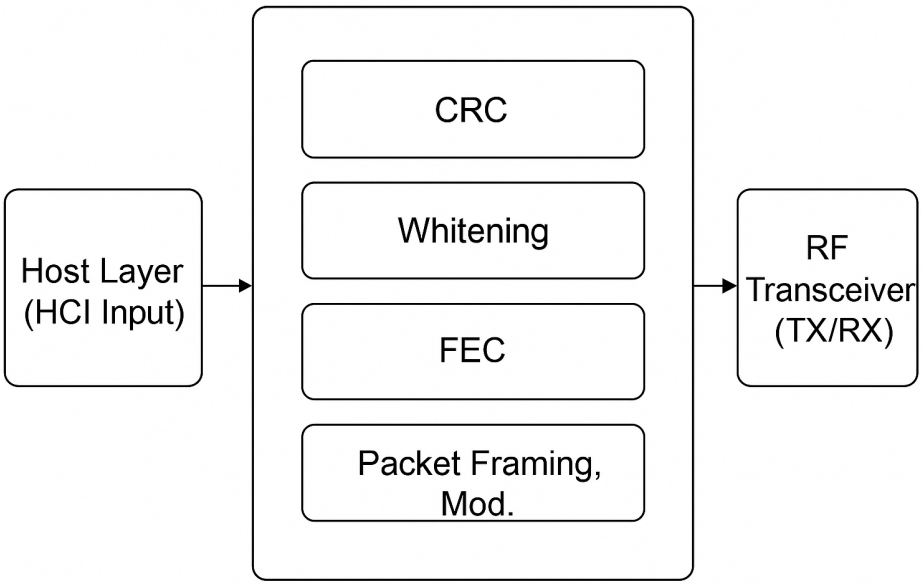


Figure 2.1: Generalized block diagram of a Bluetooth Low Energy (BLE) digital baseband architecture highlighting common functional modules reviewed in literature.

2.2 A Low-Power BLE Baseband Transmission Data Processing Module (2024)

A low-power BLE baseband transmission data processing module based on Verilog HDL was proposed by Liang Ding et al. (2024). The module comprised CRC generator, data whitener, and FEC modules utilizing the concept of modularity. The simulation test using Synopsys VCS proved functionality and low power consumption capabilities. It was shown that the use of modularity in designing the baseband system allows for better scalability and adherence to BLE 5.0 standards.

2.3 A Baseband Processor for Impulse Ultra-Wideband Communications (2005)

Blazquez et al. (2005) suggested a baseband processor design for impulse Ultra-Wide Band (UWB) communication that uses 0.18 μm CMOS technology. The baseband processor included a 4-bit, 1.2 Gsps Flash ADC, a Phase-Locked Loop (PLL), and digital synchronization circuits. With the transfer of synchronization and correlation

operations to the digital domain, the 4 architecture reduced the analog part of the circuitry. While intended for use in UWB, the approach can be employed in the design of BLE baseband processors.

2.4 A 10 mW Bluetooth Low-Energy Transceiver with On-Chip Matching (2016)

In a paper titled "A Compact 10mW GFSK Modulated Bluetooth Low Energy Transceiver with Integrated Digital Baseband" published in 2016 by J.H. Lee et al., a 10mW BLE transceiver comprising both analog and digital baseband was discussed. In their design, impedance matching on chip, GFSK modulation technique, and integrated digital baseband were some notable features. It was shown that the optimization of both RF and baseband subsystems is highly beneficial for system performance.

2.5 Comparison of Reviewed Works

In general, all these papers show advances in the design of BLE baseband. Each paper concentrated on a specific area—energy efficiency, scalability, or integration. The following table shows the major findings and limitations of each paper.

Table 2.1: Comparison of Reviewed Research Papers

Sr. No.	**Title of Paper**	**Year**	**Benefits**	**Limitations**
1	A Low-Power BLE Baseband Transmission Data Processing Module	2024	Efficient RTL design; includes CRC, whitening, and FEC modules; validated through simulation	Focused only on digital processing; lacks full-chip hardware realization
2	A Baseband Processor for Impulse Ultra-Wideband Communications	2005	Fully digital synchronization; high-speed operation with scalable architecture	High power usage for BLE standards; not BLE-specific
3	A 10 mW Bluetooth Low-Energy Transceiver with On-Chip Matching	2016	Integrated transceiver and baseband design; low-power operation suitable for IoT	Limited flexibility for custom baseband extensions

Chapter 3

Theoretical Background and Design Methodology

3.1 Theoretical Background

Bluetooth low-energy (BLE) is a radio transmission system for short-range communication that consumes minimum power and is mainly used for transmitting data. The system utilizes a wireless frequency range known as the 2.4GHz Industrial, Scientific, and Medical (ISM) bands for effective data transmission with minimum energy loss. The system is popular because of its reliability, low price and efficient use in battery-powered gadgets.

Data transmitted via BLE communication involves packet-based transmission. BLE data packet consists of a synchronization sequence known as the preamble, an access address, a header, data payload, and a Cyclic Redundancy Check (CRC). The organization of data in packets guarantees reliable data transmission by making sure that data is structured properly before being sent out.

In order to ensure high-quality data transmission, BLE applies data whitening through Linear Feedback Shift Registers (LFSRs). BLE data transmission utilizes Gaussian Frequency Shift Keying (GFSK) modulation that transforms the digital signals to analog waveforms and thus reduces interference during transmission.

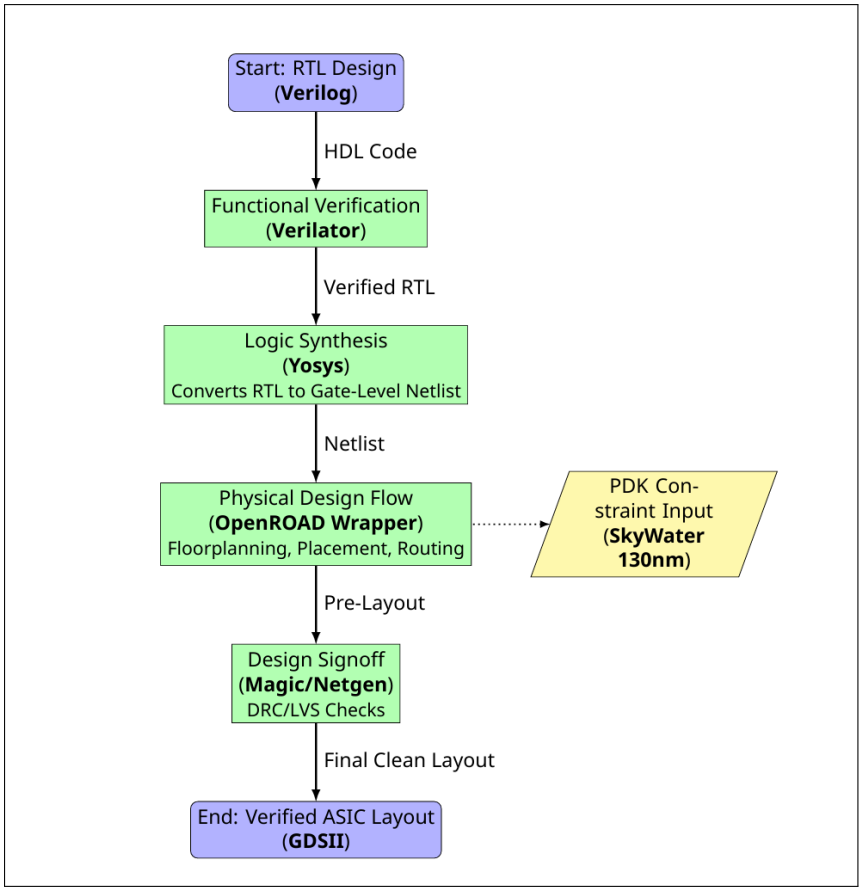


Figure 3.1: Process flow of BLE baseband processor.

3.2 Design Methodology

The architecture uses the modular development concept for hardware, which starts with RTL coding, then moves to simulation and verification. Modules are then tested individually before integrating the system. BLE digital baseband processor has two major components namely transmitter path and receiver path.

3.2.1 BLE Transmitter

Transmitter Path encodes the input data that is received by the Baseband layer through the HCI (Host Controller Interface). Working of each functional module has been explained in detail as following:

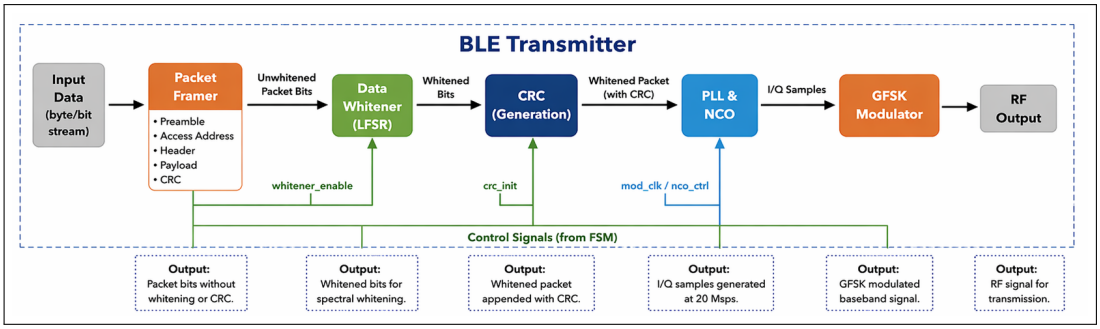


Figure 3.2: Block Diagram of BLE Transmitter

- **Input Data:** Gives the input data stream which is going to be transmitted.
- **Packet Framer:** Constructs BLE packets using preamble, access address, header, and data.
- **Data Whitener:** Uses the LFSR to scramble the bit pattern so that there is no long run of similar bits.
- **CRC Generator:** Calculates the CRC value of 24-bits and attaches it to the end of the packet for error checking.
- **PLL and NCO:** PLL(Phase Lock Loop) is used to increase the clock rate, and NCO(Numerical Controlled Oscillator) creates the frequency elements in digital I/Q format.
- **GFSK Modulator:** Gaussian Frequency Shift Keying modulates the I/Q elements to frequency changes with continuous phase to generate the RF signal.
- **Control Signals:** Controls all the blocks involved in transmitting BLE signals.

3.2.2 BLE Receiver

The BLE receiver extracts the original data from the incoming RF signal. This involves demodulation, synchronization, and validation of the data. The block diagram of the entire receiver design is illustrated below in Fig. 3.3.

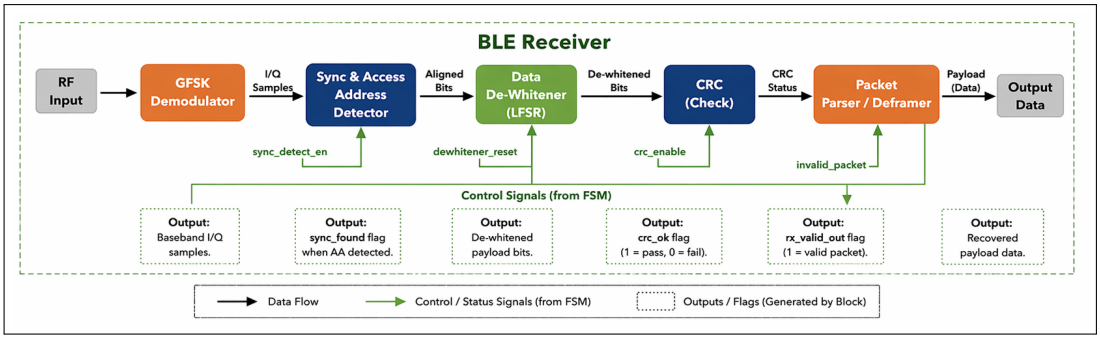


Figure 3.3: Block Diagram of BLE Receiver

- **GFSK Demodulator:** Receives the RF input and translates to digital samples using frequency changes.
- **Synchronization and Access Address Detector:** Checks if there is a valid packet and gives *sync_found* signal.
- **Data De-Whitener:** Undoes the whitening algorithm applied on the transmitted data using the LFSR sequence.
- **CRC Check:** Checks whether the transmitted data is correct through comparison of the CRC received and calculated, generates *crc_ok* signal.
- **Packet Parser:**Isolates and extracts the packet payload data.
- **Output Signals:** Generates *rx_valid_out* for valid data and *invalid_packet* for erroneous or invalid packets.
- **Control Signals:** Manages and Controls the synchronization process and data flow.

The Hardware Interface of Transceiver is thoroughly specified in two important tables. The first table (Table 3.1) gives information on the highest level I/Os of the Open Core, which provides details about the connections related to power, clock, reset, and data buses. After that, the second table (Table 3.2) describes the pin specifications of various modules inside the transmitter, such as CRC-24 Generator, Packet Framer, Data Whitener, and GFSK Modulator.

Table 3.1: Pin specifications for the BLE Baseband Processor Core.

Block Name	Pin Name	Direction	Type	Size (bits)	Description
BLE Baseband Processor Core	VDD, VSS	Input	Power	1	Power (1.8V) and Ground connections.
	clk	Input	Logic	1	Master clock input (Target: 20 MHz).
	rst_n	Input	Logic	1	Active-low asynchronous reset.
	tx_data_in	Input	Bus	8	8-bit bus to load transmit payload data.
	tx_valid_in	Input	Logic	1	Indicates tx_data_in is valid (handshake).
	start_tx	Input	Logic	1	Pulse to begin a transmission.
	demod_in	Input	Logic	1	Single-bit input from the ADC/RF front-end.
	tx_ready_out	Output	Logic	1	Indicates core is ready for new transmit data.
	rx_data_out	Output	Bus	8	8-bit bus for received payload data.
	rx_valid_out	Output	Logic	1	Indicates valid data is on rx_data_out.
	irq_tx_done	Output	Logic	1	Interrupt indicating transmission is complete.
	irq_rx_done	Output	Logic	1	Interrupt indicating a packet has been received.
	modulated_out	Output	Logic	1	Single-bit modulated GFSK output to DAC.

Table 3.2: Pin specifications for Transmitter Sub-blocks.

Block Name	Pin Name	Direction	Type	Size (bits)	Description
CRC-24 Generator	clk	Input	Logic	1	Master clock signal.
	rst_n	Input	Logic	1	Active-low asynchronous reset.
	enable	Input	Logic	1	Enables the register to process one bit.
	data_in	Input	Logic	1	A single serial data bit from the PDU.
	crc_out	Output	Bus	24	The final parallel CRC value after processing.
Packet Framer	clk	Input	Logic	1	Master clock signal.
	rst_n	Input	Logic	1	Active-low asynchronous reset.
	start_tx	Input	Logic	1	Single-cycle pulse to begin transmission.
	tx_data_in	Input	Bus	8	Parallel payload data from the host CPU.
	tx_valid_in	Input	Logic	1	Handshake signal indicating tx_data_in is valid.
	crc_in	Input	Bus	24	The completed CRC value from the CRC block.
	tx_ready_out	Output	Logic	1	Handshake signal indicating it's ready for data.

Table 3.2 – Continued

Block Name	Pin Name	Direction	Type	Size (bits)	Description
	irq_tx_done	Output	Logic	1	Interrupt signaling that transmission is complete.
	crc_enable	Output	Logic	1	Internal control signal to enable the CRC block.
	whitener_enable	Output	Logic	1	Internal control signal to enable the Whitener.
	channel_ind_out	Output	Bus	6	The 6 bits that tell the data whitener the channel
	serial_data_out	Output	Logic	1	Serialized data going to the Whitener.
Data Whitener	clk	Input	Logic	1	Master clock signal.
	rst_n	Input	Logic	1	Active-low asynchronous reset.
	enable	Input	Logic	1	Enables the whitening operation on data_in.
	data_in	Input	Logic	1	A single serial data bit from the Framer.
	channel_index	Input	Bus	6	6 digit channel number
	whitened_data_out	Output	Logic	1	The scrambled output data bit.
GFSK Modulator	clk	Input	Logic	1	Master clock signal.
	rst_n	Input	Logic	1	Active-low asynchronous reset.
	data_in	Input	Logic	1	The whitened data bit from the Whitener.

Table 3.2 – Continued

Block Name	Pin Name	Direction	Type	Size (bits)	Description
	modulated_out	Output	Logic	1	Final single-bit modulated output for the DAC.

3.3 CRC Generation and Data Whitening Logic

The CRC and data whitening modules constitute the fundamental blocks of the digital baseband scheme. Both are realized by means of LFSRs, providing an efficient implementation for the serial processing of bits.

3.3.1 CRC Generation using Fibonacci LFSR

The Cyclic Redundancy Check (CRC) is a critical component of the digital baseband, responsible for detecting transmission errors in the BLE packet. This section details the mathematical and hardware derivation of the 24-bit CRC used in the Physical Layer.

Mathematical Model in GF(2)

The CRC algorithm treats the input bitstream as a polynomial $M(x)$ and performs division over the Galois Field $GF(2)$. The generator polynomial $P(x)$ specified by the Bluetooth Core Specification is:

$$P(x) = x^{24} + x^{10} + x^{9} + x^{6} + x^{4} + x^{3} + x + 1 \tag{3.1}$$

In $GF(2)$, addition and subtraction are replaced by the XOR operation, and there are no carries. The CRC generation is defined as the remainder $R(x)$ of the following division:

$$R(x) = [M(x) \cdot x^{24}] \pmod{P(x)} \tag{3.2}$$

Step-by-Step LFSR Hardware Mapping

To implement this division efficiently in RTL, a Linear Feedback Shift Register (LFSR) in the Fibonacci configuration is utilized. The hardware mapping follows these logical steps:

1. **Register Chain:** 24 D-type flip-flops are connected in series to store the current state of the remainder.
2. **Feedback Logic:** XOR gates are placed at the feedback taps defined by the coefficients of $P(x)$.
3. **Serial Processing:** The payload bits are shifted into the LFSR bit-serially. Each shift represents one step of the long division process.
4. **Initialization:** Prior to processing, the LFSR is pre-set to a 24-bit *CRC Initialization Vector* (Seed) to ensure channel-specific synchronization.

Significance in RTL Design

The LFSR-based CRC architecture provides a zero-latency calculation of the checksum. By calculating the CRC as the packet is being transmitted or received, the baseband processor can append or verify the checksum without buffering the entire payload. This bit-serial approach minimizes the gate count and power consumption of the digital logic, which is a primary design constraint for Bluetooth Low Energy hardware.

3.3.2 Data Whitening using Galois LFSR

Data whitening is a fundamental randomization process used in the BLE Physical Layer to ensure spectral stability and facilitate receiver synchronization.

Problem Statement: DC Bias and Clock Drift

Standard digital payloads often contain repetitive sequences of "0"s or "1"s. In the RF domain, these sequences lead to a lack of transitions, causing two primary issues:

1. **DC Offset:** Continuous identical bits create a non-zero average voltage in the baseband, complicating receiver threshold detection.

2. **Clock Recovery:** Receivers utilize bit transitions to maintain timing synchronization. A lack of transitions leads to sampling jitter and increased Bit Error Rate (BER).

The Galois LFSR Mathematical Model

The whitening process utilizes a 7-bit Linear Feedback Shift Register (LFSR) configured in a Galois arrangement. The pseudo-random sequence is generated based on the following characteristic polynomial:

$$P(x) = x^7 + x^4 + 1 \tag{3.3}$$

This polynomial produces a maximal-length sequence (m-sequence) of 127 bits. The whitening is achieved by performing a bit-wise XOR operation between the raw payload D_{in} and the output of the LFSR W_{out}:

$$D_{whitened} = D_{in} \oplus W_{out} \tag{3.4}$$

Hardware Architecture and State Transitions

In a Galois LFSR, the feedback logic is distributed between the shift registers. For the polynomial $x^7 + x^4 + 1$, the state transition of the i-th register S_i at time $n+1$ is defined as:

$$S_4(n+1) = S_3(n) \oplus S_6(n) \tag{3.5}$$

All other registers S_i (where $i \neq 4$ and $i \neq 0$) simply perform a shift operation $S_i(n+1) = S_{i-1}(n)$, while $S_0(n+1) = S_6(n)$.

Significance in RTL and System Design

The Galois configuration is chosen for its superior timing characteristics in RTL synthesis, as it minimizes the combinational logic depth between registers. Furthermore, the whitening module is initialized at the start of every packet with a 7-bit seed based on the channel index. This ensures that the physical signal varies across different frequencies, providing a form of processing gain that improves the robustness of the BLE link in crowded environments.

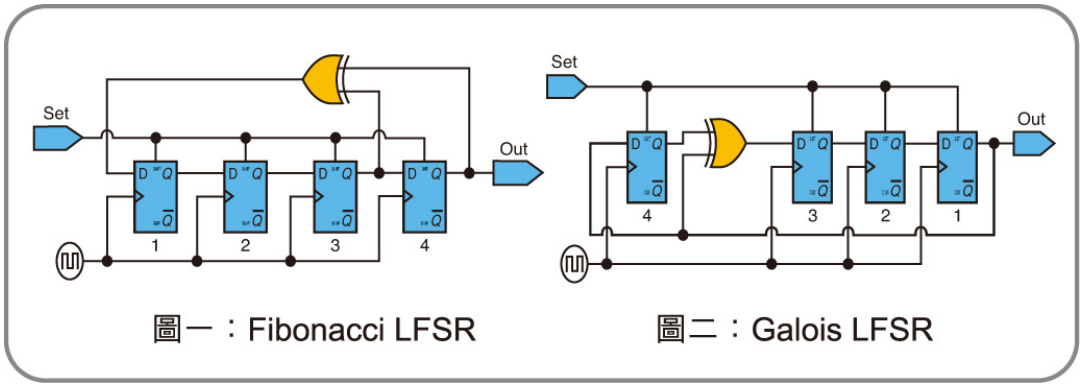


Figure 3.4: 4-bit Fibonacci and Galois Linear Feedback Shift Register (LFSR) models used for CRC generation and data whitening respectively.

3.4 3.4 Gaussian Frequency Shift Keying (GFSK) Modulation

Gaussian Frequency Shift Keying (GFSK) serves as the fundamental modulation scheme for the Bluetooth Low Energy (BLE) physical layer. It is a constant-envelope modulation technique that optimizes spectral efficiency through pre-modulation pulse shaping.

3.4.1 Principles of GFSK Operation

GFSK modulation functions by shifting the carrier frequency f_c in response to digital data. Unlike standard Frequency Shift Keying (FSK), where the frequency transitions are abrupt, GFSK utilizes a Gaussian low-pass filter to smooth the NRZ (Non-Return-to-Zero) bitstream. This filtering process suppresses the high-frequency harmonics of the square-wave pulses, thereby constraining the signal within the allocated 1 MHz channel bandwidth and minimizing Adjacent Channel Interference (ACI).

3.4.2 The Gaussian Shaping Filter

The performance of the GFSK modulator is governed by the Bandwidth-Time (BT) product. Per the BLE specification, a BT of 0.5 is employed. This value ensures a critical balance between:

1. **Occupied Bandwidth:** Reducing the power spectral density (PSD) sidelobes to satisfy regulatory masks.

2. **Intersymbol Interference (ISI):** Maintaining sufficient pulse integrity to allow for low-complexity non-coherent detection at the receiver.

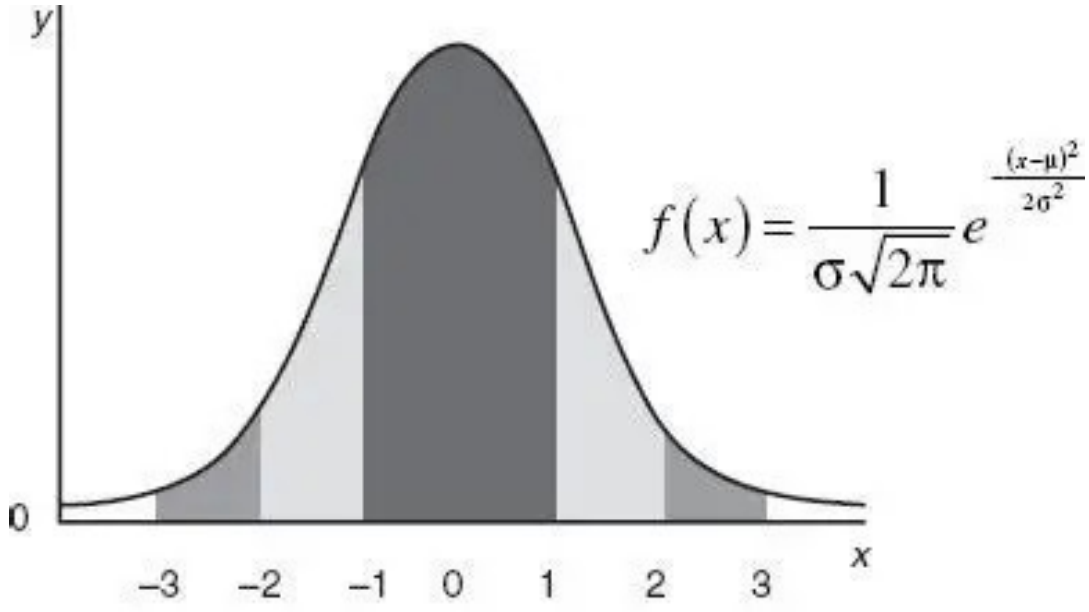


Figure 3.5: A Gaussian curve

3.4.3 Significance in RTL Implementation

The GFSK module is implemented as a digital baseband processor that drives a Numerically Controlled Oscillator (NCO). By utilizing a Look-Up Table (LUT) for the Gaussian-shaped pulses and a phase accumulator for integration, the hardware achieves a precise, phase-continuous output. This architecture is highly efficient for ASIC and FPGA implementations, providing the necessary spectral containment for BLE while minimizing power consumption.

3.5 GFSK Demodulation

GFSK demodulation is the process of recovering digital data from the received modulated signal. It detects frequency shifts in the incoming signal and converts them back into binary data. This process reverses the modulation operation performed at the transmitter. Accurate demodulation is essential for correct data recovery.

3.6 De-Whitening

De-whitening is performed at the receiver to restore the original data sequence. It uses the same Linear Feedback Shift Register (LFSR) configuration as used during data whitening. This ensures that the randomized data is converted back to its original form. Proper synchronization of the LFSR is necessary for correct de-whitening.

3.7 Packet Structure and Parsing

BLE communication uses a structured packet format for data transmission. Each packet includes a preamble, access address, header, payload, and CRC. At the receiver, the packet parser identifies and extracts these fields based on predefined positions. This allows correct interpretation and processing of the received data.

3.8 CRC and Error Detection

Cyclic Redundancy Check (CRC) is used for error detection in BLE communication. A CRC value is generated at the transmitter and appended to the packet. At the receiver, the CRC is recalculated and compared with the received value. If both values match, the data is considered valid; otherwise, it is rejected.

3.9 Receiver Operation

The receiver performs multiple operations to recover the transmitted data. It first demodulates the received signal to obtain the bitstream. The data is then de-whitened and passed to the packet parser. Finally, CRC verification ensures the correctness of the received data.

3.10 Mathematical Derivation of the GFSK Shaping Pulse

In a digital circuit, a bit is represented by a square wave. However, the "infinitely sharp" corners of a square wave require infinite bandwidth, which would cause

interference. To ensure reduced bandwidth and continuous phase for Bluetooth Low Energy (BLE), we "round off" these transitions using a **Gaussian Filter** through a process called **Convolution**.

Table 3.3: System Nomenclature and Mathematical Parameters

Symbol	**Term**	**Value**	**Definition & Significance**
f_c	Carrier Frequency	Hz	The center frequency of the RF channel.
T_b	Bit Duration	$1\,\mu$s	Time period for a single bit in a 1 Mbps PHY.
B	3dB Bandwidth	500 kHz	Frequency at which filter power drops by half.
BT	Bandwidth-Time Product	0.5	Normalized parameter defining Gaussian "smoothness".
h	Modulation Index	≈ 0.5	Ratio of frequency swing to the symbol rate.
Δf	Frequency Deviation	$\pm$250 kHz	Max frequency offset from carrier for '1' or '0'.
a	Gaussian Alpha	$\frac{\ln(2)}{2B^2}$	Constant defining the width of the Gaussian curve.
α	Time-domain Spread	$\sqrt{\frac{\ln(2)}{2}}\frac{1}{B}$	Relates filter bandwidth to impulse response.

Table 3.3: System Nomenclature and Mathematical Parameters

Symbol	**Term**	**Value**	**Definition & Significance**
$H(f)$	Frequency Response	Unitless	The filter's behavior in the frequency domain.
$h(t)$	Impulse Response	Unitless	The time-domain "bell curve" of the Gaussian filter.
$p(t)$	Rectangular Pulse	Unitless	The raw digital bit representation (NRZ).
$g(t)$	Shaping Pulse	Unitless	Result of convolving the bit with the Gaussian filter.
$\text{erf}(x)$	Error Function	Unitless	Integral of Gaussian distribution for pulse calculation.
$s(t)$	Transmitted Signal	V / W	The final continuous-time RF waveform equation.
$\theta(t)$	Instantaneous Phase	Radians	Time-varying phase accumulated by the NCO.
a_i	Data Symbol	$\{+1, -1\}$	i-th bit of payload mapped to polar levels.
$GF(2)$	Galois Field 2	$\{0, 1\}$	Mathematical field where XOR is addition/subtraction.
$M(x)$	Message Polynomial	Polynomial	The bitstream treated as a mathematical expression.

Table 3.3: System Nomenclature and Mathematical Parameters

Symbol	**Term**	**Value**	**Definition & Significance**
$P(x)$	Generator Polynomial	Polynomial	Fixed "divisor" used for CRC or Whitening logic.
$R(x)$	Remainder Polynomial	24-bit	The calculated CRC checksum (Syndrome).
$Q(x)$	Quotient Polynomial	Discarded	The result of the division, unused in hardware.
$T(x)$	Codeword	Polynomial	The complete packet (Message + CRC).
S_k	LFSR State Vector	7/24-bit	Current binary value stored in the flip-flops.
T	Transition Matrix	Matrix	Mathematical model of LFSR feedback connections.
E_b/N_0	SNR per bit	dB	Ratio of signal energy to noise power density.
P_b	Bit Error Rate (BER)	Probability	Theoretical likelihood of bit reception error.

3.10.1 Defining the Gaussian Filter in Frequency

We define the filter in the Frequency Domain (f) as a bell curve:

$$H(f) = e^{-af^2} \tag{3.6}$$

To find the constant a, we use the **3dB Bandwidth (B)**. At frequency $f = B$, the voltage drops to $1/\sqrt{2}$:

1. $e^{-aB^2} = \frac{1}{\sqrt{2}}$
2. $-aB^2 = \ln(1) - \ln(\sqrt{2}) = -\frac{1}{2}\ln(2)$
3. $a = \frac{\ln(2)}{2B^2}$

The final Frequency Response is:

$$H(f) = \exp\left(-\frac{\ln(2)}{2}\left(\frac{f}{B}\right)^2\right) \tag{3.7}$$

3.10.2 The Inverse Fourier Transform

To implement this in RTL, we need the **Impulse Response** $h(t)$ in the Time Domain. We use the Inverse Fourier Transform:

$$h(t) = \int_{-\infty}^{\infty} H(f)e^{j2\pi ft}df = \int_{-\infty}^{\infty} e^{-af^2}e^{j2\pi ft}df \tag{3.8}$$

By completing the square in the exponent and solving the Gaussian integral, we obtain:

$$h(t) = \frac{\sqrt{\pi}}{\alpha}\exp\left(-\frac{\pi^2 t^2}{\alpha^2}\right) \quad \text{where} \quad \alpha = \sqrt{\frac{\ln(2)}{2}}\frac{1}{B} \tag{3.9}$$

3.10.3 Convolution and the Error Function (erf)

The shaping pulse $g(t)$ is the convolution of a rectangular bit pulse $p(t)$ (of duration T_b) with the filter $h(t)$:

$$g(t) = p(t) * h(t) = \frac{1}{T_b}\int_{t-T_b/2}^{t+T_b/2} h(\tau)d\tau \tag{3.10}$$

Since the integral of a Gaussian has no elementary solution, we use the **Error Function**, defined as $\text{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x e^{-u^2}du$. After substitution and introducing the BT **Product**, the final engineering form for the BLE Baseband is:

$$g(t) = \frac{1}{2T_b}\left[\text{erf}\left(\pi\sqrt{\frac{2}{\ln 2}}BT\left(\frac{t}{T_b}+\frac{1}{2}\right)\right) - \text{erf}\left(\pi\sqrt{\frac{2}{\ln 2}}BT\left(\frac{t}{T_b}-\frac{1}{2}\right)\right)\right] \tag{3.11}$$

3.10.4 Hardware Significance (RTL Implementation)

For the BLE project, calculating erf in real-time is computationally expensive for hardware. Instead:

- We fix $BT = 0.5$ and $T_b = 1\,\mu s$ as per the standard.
- We pre-calculate values of $g(t)$ and store them in a **Look-Up Table (LUT)**.
- The RTL logic reads the LUT to drive the GFSK modulator, ensuring a perfect smooth frequency shift.

3.11 Phase Accumulator Calculations

To analyze the conversion of digital bits into a Radio Frequency (RF) waveform, the following parameters are defined:

- **Carrier Frequency (f_c):** The center frequency (e.g., 2.402 GHz).
- **Modulation Index (h):** For BLE, $h = 0.5$.
- **Bit Duration (T_b):** The time for one bit ($1\,\mu s$ for 1 Mbps PHY).
- **Instantaneous Phase ($\theta(t)$):** The value used to drive the Sine/Cosine generator.

3.11.1 Derivation of Frequency Deviation (Δf)

The frequency deviation is the maximum distance the signal moves from the center carrier. In GFSK, this is derived from the modulation index h.

$$h = \text{Total Frequency Swing} \times T_b = (2\Delta f) \cdot T_b \tag{3.12}$$

Isolating Δf through algebraic division:

$$\Delta f = \frac{h}{2T_b} \tag{3.13}$$

Substituting the BLE standard values:

$$\Delta f = \frac{0.5}{2 \times 10^{-6}} = 250\,\text{kHz} \tag{3.14}$$

3.11.2 Phase Integral Derivation

The Baseband Processor generates a phase $\theta(t)$ that is the integral of the frequency. Given a sequence of bits a_i and the shaping pulse $g(t)$, the instantaneous frequency deviation is:

$$f_{\text{dev}}(\tau) = \sum_i a_i g(\tau - iT_b) \tag{3.15}$$

The phase is the integral of this frequency deviation over time:

$$\theta(t) = 2\pi h \int_{-\infty}^{t} \left[\sum_i a_i g(\tau - iT_b) \right] d\tau \tag{3.16}$$

Using the linearity property of integration, the summation is moved outside the integral:

$$\theta(t) = 2\pi h \sum_i a_i \left(\int_{-\infty}^{t} g(\tau - iT_b) d\tau \right) \tag{3.17}$$

This confirms that the total phase is the cumulative sum of the areas under the shaped pulses, ensuring a smooth transition between symbols.

3.11.3 RTL Significance: From Calculus to Hardware

In the RTL implementation, this mathematical model is mapped to a **Phase Accumulator** within a Numerically Controlled Oscillator (NCO):

1. **Continuous vs. Discrete:** The integral is replaced by a digital summation: $\theta[n] = \theta[n-1] + f_{word}[n]$.

2. **Phase Continuity:** The accumulator ensures that when a bit changes (e.g., $+1$ to -1), the phase starts from its last value rather than resetting. This prevents "Phase Jumps" that would cause spectral splatter.

3. **Hardware Efficiency:** The integral of the shaping pulse is pre-calculated and stored in a Look-Up Table (LUT), reducing the FPGA/ASIC workload to simple addition.

3.12 Implementation Parameters and NCO Calibration

To translate the theoretical GFSK model into a functional RTL design, the system parameters must be mapped to the hardware clocking constraints and Numerically Controlled Oscillator (NCO) bit-widths.

3.12.1 Clock Synthesis and PLL Scaling

The design utilizes a Phase-Locked Loop (PLL) to multiply the input reference frequency, providing sufficient timing resolution for the GFSK shaping pulse.

- **Input Reference (F_{ref}):** 20 MHz
- **Target Internal Clock (F_{int}):** 400 MHz
- **Multiplication Factor (M):**

$$M = \frac{F_{int}}{F_{ref}} = \frac{400 \text{ MHz}}{20 \text{ MHz}} = 20 \tag{3.18}$$

In the Verilog module `tx_modulator_clocks.v`, this is parameterized as `PLL_MULT = 20`.

3.12.2 Samples Per Symbol (SPS)

The SPS defines the discretization of the Gaussian ramp. A higher SPS results in a smoother frequency transition and lower spectral leakage.

- **Data Rate (R):** 20 Mbps
- **SPS Calculation:**

$$SPS = \frac{F_{int}}{R} = \frac{400 \text{ MHz}}{20 \text{ Mbps}} = 20 \text{ samples/bit} \tag{3.19}$$

3.12.3 NCO Tuning Word Calculation

The NCO employs an 8-bit phase accumulator ($N = 256$). The frequency control word (FCW or M) determines the phase step size per clock cycle:

$$M = \frac{F_{target} \times 2^N}{F_{int}} \tag{3.20}$$

Carrier and Deviation Mapping

Using the 400 MHz system clock, the tuning words for the center carrier and the frequency shift are calculated as:

- **Center Carrier (100 MHz):** $M_{center} = \frac{100 \times 256}{400} = 64$ ($8'h40$)
- **Frequency Deviation (10 MHz):** $M_{dev} = \frac{10 \times 256}{400} \approx 6$

Consequently, the modulated output frequencies are represented by the following digital values:

- **Logic '1' (110 MHz):** $64 + 6 = 70$
- **Logic '0' (90 MHz):** $64 - 6 = 58$

3.12.4 Simulation and Verification Results

The simulation timing is scaled to match the 400 MHz physical behavior. With a period $T = 2.5$ ns, the simulator toggle rate is set to 1.25 ns. The following table summarizes the expected versus observed values during GTKWave verification.

Table 3.4: GTKWave Verification Summary

Signal	**Expected Frequency**	**Expected Period**
`ext_clk`	20 MHz	50 ns
`clk_fast`	400 MHz	2.5 ns
`rf_out` (Logic 1)	106.4 MHz*	$\approx$ 9.4 ns
`rf_out` (Logic 0)	93.6 MHz*	$\approx$ 10.6 ns

3.13 Finite State Machine (FSM) Implementation

Coordination of the BLE baseband happens through the control of two main FSMs. The controllers take care of time management for data serialization, whitening, and validation, thus ensuring that the hardware meets the temporal needs of the link layer.

3.13.1 Transmitter FSM Logic

Transmitter FSM controls the formatting and construction of BLE packets from raw data stream to frame.

Table 3.5: Transmitter State-Transition and Control Mapping

Pin Name	IDLE	PREAMBLE	ADDR	PDU	CRC_SUM	DONE
`tx_ready_out`	0	0	0	1	0	0
`crc_enable`	0	0	0	1	0	0
`whitener_enable`	0	0	0	1	1	0
`serial_data_out`	0	Preamble	Addr	Data	CRC	0
`irq_tx_done`	0	0	0	0	0	1

Functional Analysis of TX Control Signals

- **Data Gating (`crc_enable`):** This particular signal is high only in the PDU mode. This way, the CRC generator will only receive the payload bits, ignoring the Preamble and the Access Address, as required by the Bluetooth Low Energy standard.
- **Spectrum Shaping (`whitener_enable`):** The whitener occurs active during both `PDU` and `CRC_SUM` states. Even though the CRC is derived from the data, the BLE specification demands that the CRC itself should undergo whitening to ensure DC balance and avoid spectral lines.
- **Synchronization:** The `serial_data_out` behaves like a multiplexer, which switches among a fixed preamble, the Access Address, and dynamically created CRC.

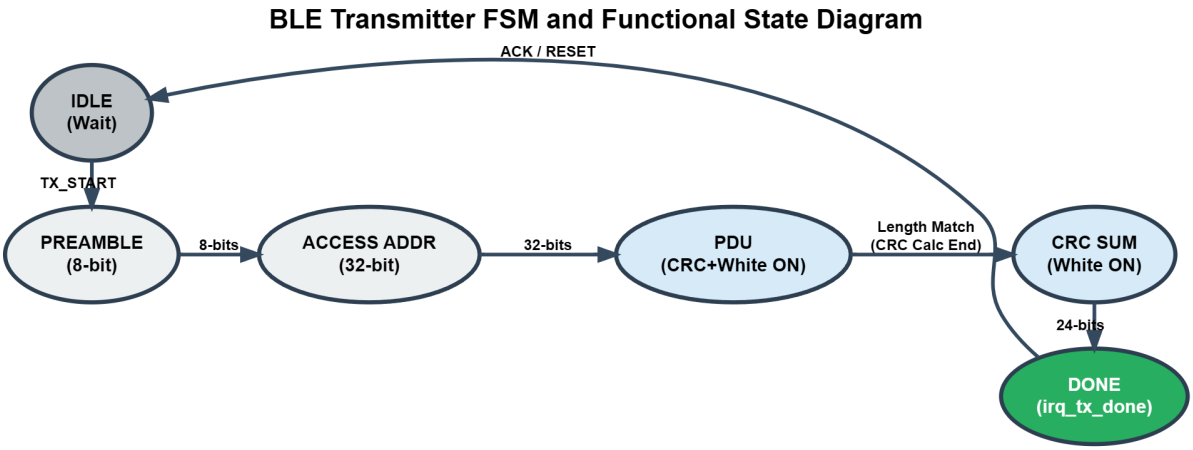


Figure 3.6: Transmitter Operational State Diagram

3.13.2 Receiver FSM Logic

The receiver FSM is a reactive system, driven by the detection of specific bit patterns in the incoming RF stream.

Table 3.6: Receiver State-Transition and Control Mapping

Pin Name	IDLE	ADDR_SYNC	HEADER	PAYLOAD	CRC_RX	CHECK	DONE
`rx_ready`	1	0	0	0	0	0	0
`rx_data_valid`	0	0	0	1	0	0	0
`crc_calc_en`	0	0	1	1	0	0	0
`crc_valid`	0	0	0	0	0	1/0	0
`packet_complete`	0	0	0	0	0	0	1
Data Bus	0	0	0	Rx Byte	0	0	0

Functional Analysis of RX Control Signals

- **Hardware Synchronization:** The FSM starts in `IDLE` with `rx_ready` high, enabling the preamble correlator. Once the Access Address is matched in `ADDR_SYNC`, the FSM establishes "Word Alignment."

- **Real-time Verification (`crc_calc_en`):** As bits arrive, they are simultaneously stored in a shift register and processed by the local CRC LFSR. This "on-the-fly" calculation is indicated by `crc_calc_en` being high during the Header and Payload phases.

- **Validation Phase (`CHECK`):** This is a zero-latency combinational check. The value of `crc_valid` is the direct result of a bitwise comparison between the received CRC bits and the locally computed remainder.
- **Data Handshake:** The `rx_data_valid` acts as a "Data Ready" strobe for the system bus, allowing the captured `Rx Byte` to be written to memory only when a complete octet has been received.

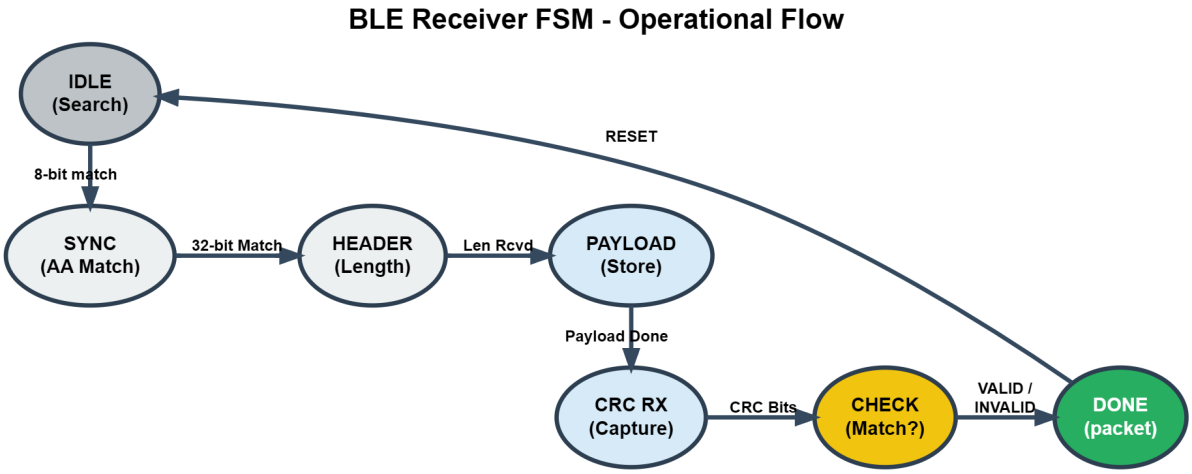


Figure 3.7: Receiver Parser State Diagram

3.14 Summary

The theoretical basis of the BLE baseband transceiver was provided in this chapter. The basics of BLE communication, such as packet, the data whitening technique using LFSR, and GFSK modulation and demodulation processes were discussed in detail.

Moreover, mathematical and system level calculations for the design process were carried out. The frequency deviation and phase generation equations for GFSK were derived for continuous phase modulation. The use of the PLL for frequency scaling and the process of the phase generation using the phase accumulator in the NCO(Numerical Control Oscillator) were provided. All these together form the connection between theoretical modeling and RTL design.

Generally, this chapter has provided sufficient information on the BLE baseband system and served as a good theoretical basis for the design described in the next chapters.

Table 3.7: Mathematical Core to RTL Logic Mapping

Module	**Mathematical Concept**	**RTL Logic Block**
Gaussian Filter	Convolution Integral ($g * h$)	Look-Up Table (LUT)
Modulator	Integration ($\int f\, dt$)	Phase Accumulator
CRC Generator	Polynomial Division ($M(x)/P(x)$)	Fibonacci LFSR
Whitening	Stochastic PRBS Generation	Galois LFSR
NCO	Sinusoidal Sampling	ROM / CORDIC Engine

Chapter 4

Software Used and Simulation Results

4.1 Software Tools Used

The design and verification of the BLE digital baseband processor were done using open-source Electronic Design Automation (EDA) tools. Each software has its unique role in the hardware design process.

- **Iverilog:** Verilog compiler that is used to verify the functionality of HDL modules.
- **Yosys:** Converts Verilog RTL code into gate-level netlist and optimizes logic for minimum area and maximum speed.
- **OpenROAD:** Place and route tool that optimizes timings.
- **Magic:** Checks DRC(Design Rule Check) rules and exports the final GDSII layout for manufacturing.
- **Netgen:** Executes and Validates the netlist and layout through LVS(Layout-Versus-Schematic).
- **Verilator:** Simulator for high-speed verification and linting of Verilog code.
- **KLayout:** Graphical viewer used for inspection and validation of GDSII layout files.

4.2 Simulation Results

The output from the simulation confirms that the design is working perfectly fine. For testing purposes, an input message is taken and the performance of both transmitter and receiver sections of the BLE baseband transceiver is investigated.

Transmitter Waveform:

Figure 4.1: Transmitter Simulation Waveform

The waveform from the transmitter represents the creation of a whole BLE data packet. This is evident from the inclusion of the preamble, access address, header, payload, and CRC in the signal.

Receiver Waveform:

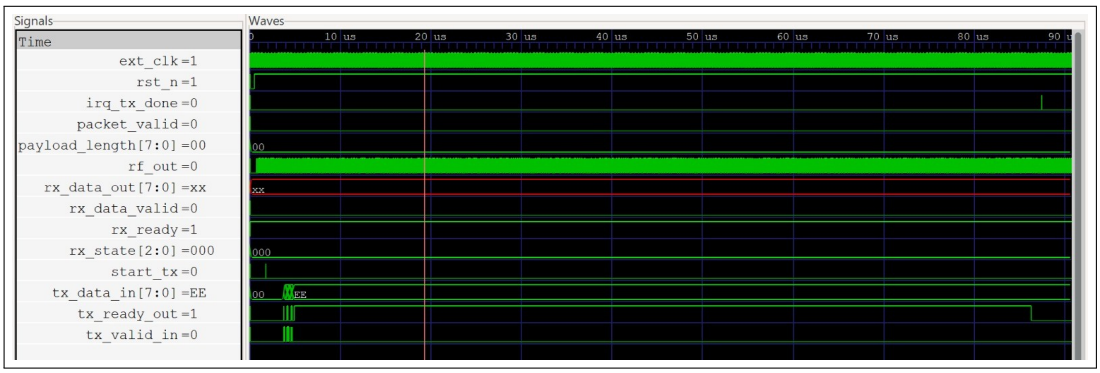


Figure 4.2: Receiver Simulation Waveform

The demodulated waveform proves the accuracy of the demodulation process along with data retrieval. The data de-whitening and packets parsing modules perform extraction of transmitted data successfully.

Terminal Output Result:

```
PS C:\Users\Abishek\Desktop\Final_year_project> vvp my_sim.vvp +MSG="This is a test message"
VCD info: dumpfile ble_system_full.vcd opened for output.
----------------------------------------
>> Terminal Input Detected: 'This is a test message'
>> Payload Size: 22 bytes
----------------------------------------

========================================
        BLE TX HARDWARE PROFILING
========================================
 Payload Sent : 'This is a test message'
 Payload Size : 22 Bytes (176 bits)
 Total Time   : 13675 ns
 Total Cycles : 269
----------------------------------------
 Cycles Spent Per FSM Stage:
   [0] IDLE     : 12
   [1] PREAMBLE : 8 (Expected ~8)
   [2] ADDR     : 32 (Expected ~32)
   [3] HEADER   : 16 (Expected ~16)
   [4] PAYLOAD  : 176 (Expected ~176)
   [5] CRC      : 24 (Expected ~24)
   [6] DONE     : 1
========================================

Testbenches/ble_tx_system_tb.v:106: $finish called at 13675000 (1ps)
```

Figure 4.3: Simulation Terminal Output.

```
PS C:\Users\Abishek\Desktop\Final_year_project> python decode_packet.py
Found 'serial_data_out' mapped to symbol '0'. Extracting waveform...
Extracted 257 raw bits. Slicing packet...

================================================
 BLE PACKET DECODER RESULTS
================================================
PREAMBLE
  TX Binary (Air):  01010101
  Logic Binary:     10101010
  Hexadecimal:      AA
------------------------------------------------
ACCESS ADDRESS
  TX Binary (Air):  01101011011111011001000101110001
  Logic Binary:     11010110101111010001001100011110
  Hexadecimal:      D6 BE 89 8E
------------------------------------------------
PDU HEADER
  TX Binary (Air):  0000000001101000
  Logic Binary:     0000000000010110
  Hexadecimal:      00 16
------------------------------------------------
PAYLOAD
  TX Binary (Air):  0010111000010110100101101100111000000100100101101100111000000100100001100000010000101110101
1011100000010010110110101001110011011101101110100001101110011010100110
  Logic Binary:     0111010001101000011010010111001100100000011010010111001100100000011000010010000001110100011
1101000010000001101101011001010111001101110011011000010110011101100101
  Hexadecimal:      74 68 69 73 20 69 73 20 61 20 74 65 73 74 20 6D 65 73 73 61 67 65
  Plain Text:       'this is a test message'
------------------------------------------------
CRC
  TX Binary (Air):  011010100111100010100001
  Logic Binary:     010101100001111010000101
  Hexadecimal:      56 1E 85
------------------------------------------------
Trailing Idle Bits: 1
PS C:\Users\Abishek\Desktop\Final_year_project>
```

Figure 4.4: Breakdown of the Packet.

Output from the terminal demonstrates the processing of the input string "This is a test message" by the system. This includes the payload size, total number of cycles, and state-machine execution of the system per stage. The output proves that packets were successfully transmitted, 24-bit CRC was generated, and transmission completed successfully.

4.3 Simulation and Performance Results

The complete BLE baseband transceiver was verified using a closed-loop simulation environment where the transmitter's RF output was directly interfaced with the receiver's RF input. The following table summarizes the architectural parameters and the performance metrics obtained from the simulation runs.

Table 4.1: System Implementation and Verification Summary

Parameter Category	**Implemented Value / Result**
System Specifications	
Baseband Data Rate	1 Mbps (Scaled for Simulation: 20 Mbps)
System Clock	20 MHz
Internal Clock (F_{int})	400 MHz (via PLL)
Architectural Features	
Data Whitening Engine	7-bit LFSR ($x^7 + x^4 + 1$)
Error Detection (CRC)	24-bit LFSR
CRC Polynomial	$x^{24} + x^{10} + x^9 + x^6 + x^4 + x^3 + x + 1$
Receiver Control Logic	7-State Finite State Machine (FSM)
Payload Support	Dynamic extraction (0 to 255 bytes)
Simulation Performance	
TX Pipeline Latency	8 clock cycles
RX Processing Latency	7 clock cycles
Verification Environment	
Simulation Engine	Icarus Verilog
Waveform Analysis	GTKWave (.vcd format)
Verified Scenarios	Preamble Sync, Address Lock, CRC Validation

4.3.1 Performance Discussion

The measured latencies (8 cycles for TX and 7 cycles for RX) indicate an extremely efficient real-time processing flow. Since the internal clock runs at 400 MHz, the processing delay is negligible compared to the 1 Mbps bit duration ($1\,\mu s$).

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

Designing and simulating of a BLE digital baseband transceiver has been successfully accomplished using Verilog HDL. The system performed critical baseband functionalities like packet generation, data whitening, GFSK modulation, demodulation, de-whitening, packet decoding, and error detection using 24-bit cyclic redundancy checks. The successful simulation confirmed that the transmitted data was correctly received at the other end through proper synchronization and data verification. This project gives an insight into how BLE communication is done internally and provides an excellent platform to understand the designing of a wireless system.

5.2 Future Scope

However, the design is currently confined to the simulation of the baseband block alone. For future studies, this design can be extended to include its implementation using FPGA technology to test its real-world performance. Its connection to the RF front end will help realize wireless communication fully. There are several ways through which power efficiency can be improved, and there can also be other additions like channel hopping and multiple nodes communication.

References

1. N. Sharma and R. Kumar, "Design of Low-Power BLE Transceiver for IoT Applications," *E3S Web of Conferences (VESSEP 2023)*, vol. 404, pp. 1–5, 2023. [Online]. Available: https://doi.org/10.1051/e3sconf/202340401044

2. C. Su and M. Lee, "A 10 mW Bluetooth Low-Energy Transceiver with On-Chip Matching," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 3, pp. 830–841, Mar. 2021. [Online]. Available: https://doi.org/10.1109/JSSC.2020.3036432

3. R. Blazquez, F. P. de Gyvez, R. D. Struik, and J. Pineda, "A 2.4 GHz Bluetooth Transceiver for Low-Power Wireless Communications," *IEEE Journal of Solid-State Circuits*, vol. 40, no. 2, pp. 474–482, Feb. 2005. [Online]. Available: https://doi.org/10.1109/JSSC.2004.839503

4. Bluetooth Special Interest Group, "Bluetooth Core Specification Version 5.4," Feb. 2023. [Online]. Available: https://www.bluetooth.com/specifications/bluetooth-core-specification/

5. C. Gomez and J. Paradells, "Overview and Evaluation of Bluetooth Low Energy: An Emerging Low-Power Wireless Technology," *Sensors*, vol. 12, no. 9, pp. 11734–11753, 2012. [Online]. Available: https://doi.org/10.3390/s120911734

6. P. Kindt, D. Yunge, R. Diemer, and S. Chakraborty, "Precise Energy Modeling for the Bluetooth Low Energy Protocol," *arXiv preprint arXiv:1403.2919*, Mar. 2014. [Online]. Available: https://arxiv.org/abs/1403.2919

7. S. Hosni, T. Zaki-Abdellatif, I. Siouane, and Y. Askouri, "Bluetooth Low Energy: A Survey," *International Journal of Computer Applications*, vol. 162, no. 1, pp. 10–18, Mar. 2017. [Online]. Available: https://doi.org/10.5120/ijca2017913248

8. B. Pang, X. Liu, B. Li, J. Yang, and Z. Tang, "Bluetooth Low Energy Interference

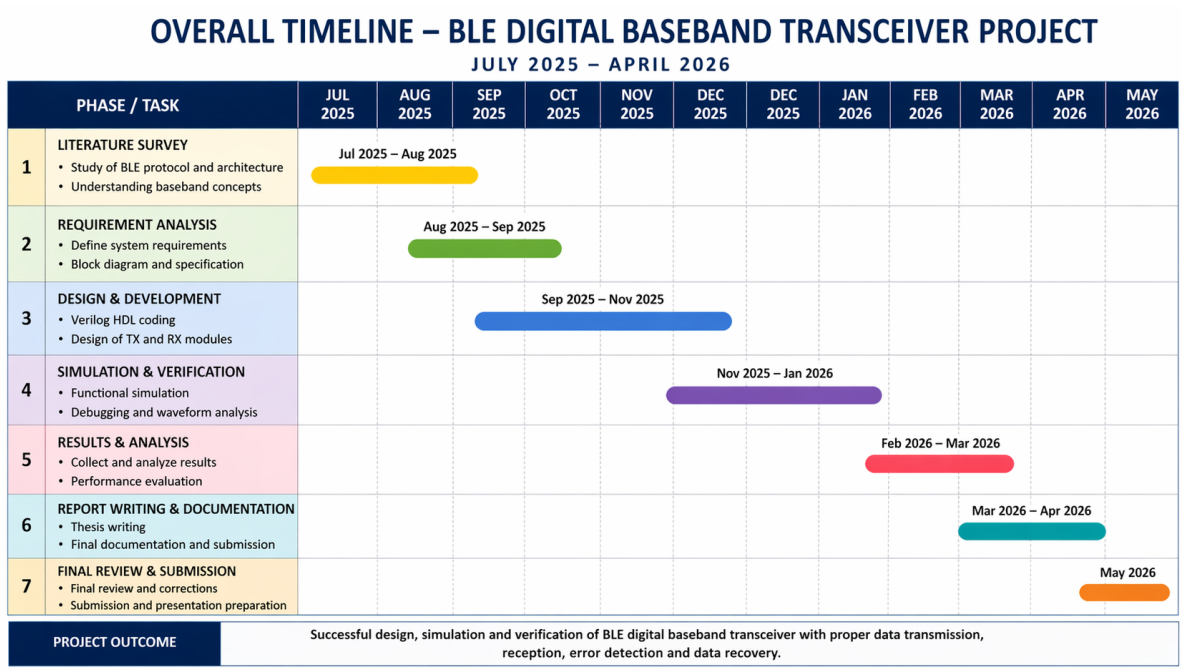
Awareness Scheme," *Sensors*, vol. 21, no. 7, p. 2257, Apr. 2021. [Online]. Available: https://doi.org/10.3390/s21072257

9. S. Sridhar, P. Misra, G. S. Gill, and J. Warrior, "CheepSync: A Time Synchronization Service for Resource-Constrained Bluetooth Low Energy Advertisers," *arXiv preprint arXiv:1501.06479*, Jan. 2015. [Online]. Available: https://arxiv.org/abs/1501.06479

10. M. Kolakowski, "Kalman Filter Based Localization in Hybrid BLE-UWB Positioning System," *arXiv preprint arXiv:2404.02349*, Apr. 2024. [Online]. Available: https://arxiv.org/abs/2404.02349

11. Bluetooth SIG, "The Bluetooth Low Energy Primer Version 1.2," Mar. 2024. [Online]. Available: https://www.bluetooth.com/wp-content/uploads/2022/05/the-bluetooth-le-primer-v1.2.0.pdf

Appendix A: Project Timeline Chart



Overall Timeline of VLSI Based BLE Digital Baseband Transceiver Project (July 2025 – April 2026)